CHAPTER 34

Regularization for Deep Learning

Regularization is a technique for reducing the variance in the validation set, thus preventing the model from overfitting during training. In doing so, the model can better generalize to new examples. When training deep neural networks, a couple of strategies exist for use as a regularizer.

Dropout

Dropout is a regularization technique that prevents a deep neural network from overfitting by randomly discarding a number of neurons at every layer during training. In doing so, the neural network is not overly dominated by any one feature as it only makes use of a subset of neurons in each layer during training. In doing so, Dropout resembles an ensemble of neural networks as a similar but distinct neural network is trained at each layer. Dropout works by designating a probability that a neuron will be dropped in a layer. This probability value is called the Dropout rate. Figure 34-1 shows an example of a network with and without Dropout.

E. Bisong, *Building Machine Learning and Deep Learning Models on Google Cloud Platform*,
https://doi.org/10.1007/978-1-4842-4470-8_34

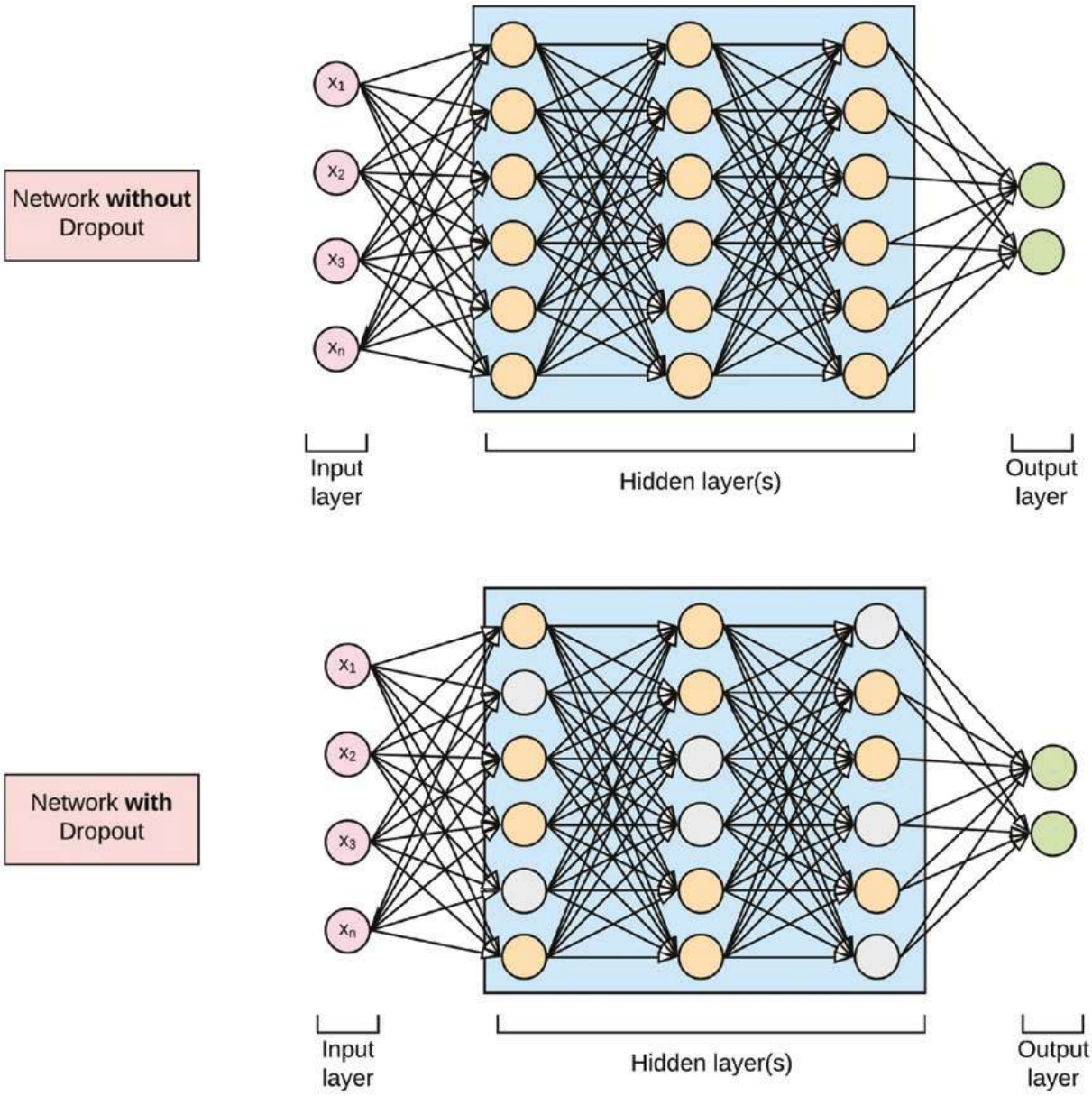

Figure 34-1. *Dropout. Top: Neural network without Dropout. Bottom: Neural network with Dropout.*

In TensorFlow 2.0 Dropout is added to a model with the method **'tf.keras.layers.Dropout()'**. The **'rate'** parameter of the method controls the fraction of the input units to drop. It is assigned a float value between 0 and 1. The following code listing shows an MLP Keras model with Dropout applied.

```
# create the model
def model_fn():
    model = tf.keras.Sequential()
    # Adds a densely-connected layer with 256 units to the model:
    model.add(tf.keras.layers.Dense(256, activation='relu', input_dim=784))
    # Add Dropout layer
    model.add(tf.keras.layers.Dropout(rate=0.2))
    # Add another densely-connected layer with 64 units:
    model.add(tf.keras.layers.Dense(64, activation='relu'))
    # Add a softmax layer with 10 output units:
    model.add(tf.keras.layers.Dense(10, activation='softmax'))

    # compile the model
    model.compile(optimizer=tf.train.AdamOptimizer(0.001),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model
```

Data Augmentation

Data augmentation is a method for artificially generating more training data points. This technique is precipitated on the observation that for an increasingly large training dataset mitigates the problem of overfitting. For some problems, it may be easy to artificially generate fake data, while for others it may not readily be the case. A classic example where we can use data augmentation is in the case of image classification. Here artificial images can easily be created by rotating or scaling the original images to create more variations of the dataset for a particular image class.

Noise Injection

The noise injection regularization method adds some Gaussian noise to the network inputs during training. Also, Gaussian noise can be added to the hidden units to mitigate overfitting. Yet still another form of injecting noise into the network is to add some Gaussian noise to the network weights. Noise injection can be considered as a form of data augmentation. The amount of noise added is a configurable hyper-parameter.

Too little noise has no effect, whereas too much noise makes the mapping function too challenging to learn.

In TensorFlow 2.0, noise injection can be added to the model as a form of data augmentation using the method **'tf.keras.layers.GaussianNoise()'**. The **'stddev'** parameter of the method controls the standard deviation of the noise distribution. The following code listing shows an MLP Keras model with Gaussian noise applied to the model.

```
# create the model
def model_fn():
    model = tf.keras.Sequential()
    # Adds a densely-connected layer with 256 units to the model:
    model.add(tf.keras.layers.Dense(256, activation='relu', input_dim=784))
    # Add Gaussian Noise
    model.add(tf.keras.layers.GaussianNoise(stddev=1.0))
    # Add another densely-connected layer with 64 units:
    model.add(tf.keras.layers.Dense(64, activation='relu'))
    # Add a softmax layer with 10 output units:
    model.add(tf.keras.layers.Dense(10, activation='softmax'))

    # compile the model
    model.compile(optimizer=tf.keras.optimizers.RMSprop(),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model
```

Early Stopping

Early stopping involves storing the model parameters each time there is an improvement in the loss (or error) estimate on the validation dataset. At the end of the training phase, the stored model parameters are used rather than the last known parameter before termination.

The technique of early stopping is based on the observation that for a sufficiently complex classifier, as the training phase progresses, the error estimate on the training data continues to decrease, whereas the validation data will see an increase in the model error measure. This is illustrated in Figure 34-2.

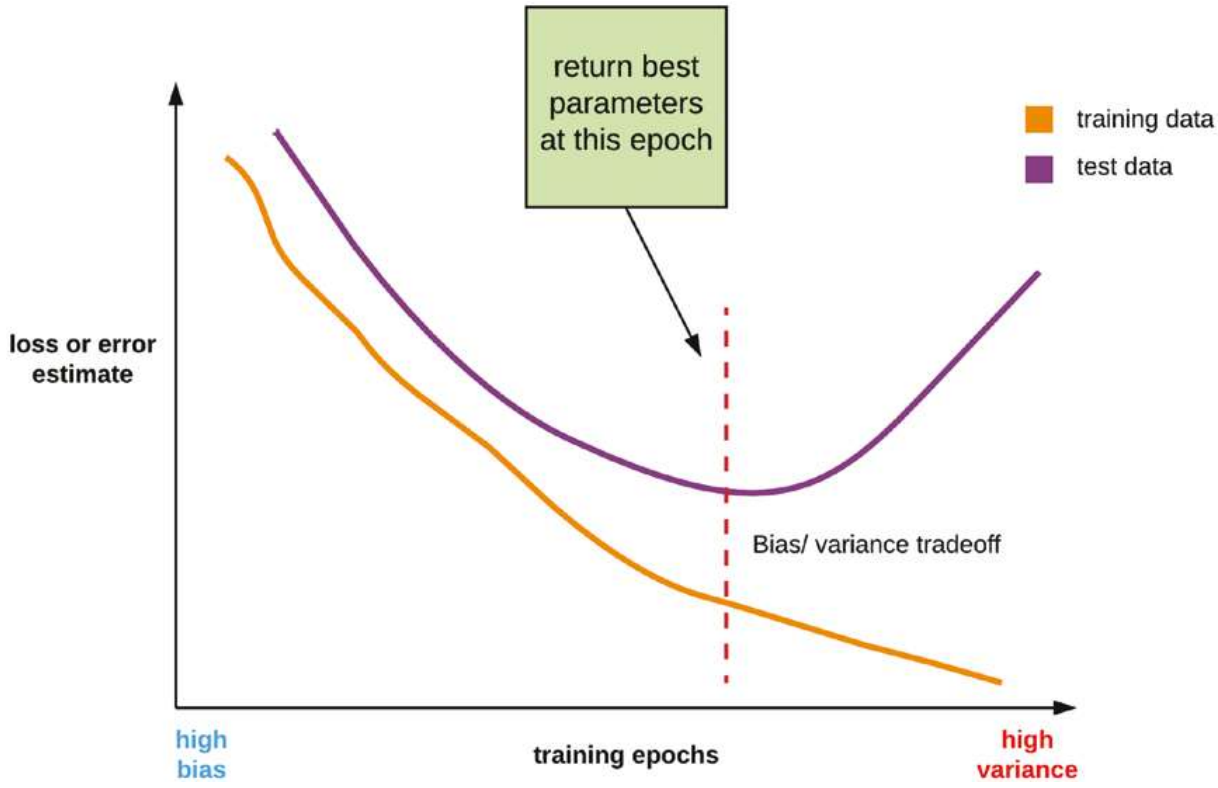

Figure 34-2. *Early stopping*

In TensorFlow 2.0, early stopping can be applied to stop training when there is no improvement in the validation accuracy or loss by applying the **'tf. keras.callbacks. EarlyStopping()'** method as a callback when training the model. For completeness sake, we will produce a complete code listing with early stopping applied to the MLP Fashion-MNIST model.

```
# install tensorflow 2.0
!pip install -q tensorflow==2.0.0-beta0

# import packages
import tensorflow as tf
import numpy as np

# import dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.
load_data()

# flatten the 28*28 pixel images into one long 784 pixel vector
```

```
x_train = np.reshape(x_train, (-1, 784)).astype('float32')
x_test = np.reshape(x_test, (-1, 784)).astype('float32')

# scale dataset from 0 -> 255 to 0 -> 1
x_train /= 255
x_test /= 255

# one-hot encode targets
y_train = tf.keras.utils.to_categorical(y_train)
y_test = tf.keras.utils.to_categorical(y_test)

# create the model
def model_fn():
    model = tf.keras.Sequential()
    # Adds a densely-connected layer with 256 units to the model:
    model.add(tf.keras.layers.Dense(256, activation='relu', input_dim=784))
    # Add another densely-connected layer with 128 units:
    model.add(tf.keras.layers.Dense(128, activation='relu'))
    # Add another densely-connected layer with 64 units:
    model.add(tf.keras.layers.Dense(64, activation='relu'))
    # Add another densely-connected layer with 32 units:
    model.add(tf.keras.layers.Dense(32, activation='relu'))
    # Add a softmax layer with 10 output units:
    model.add(tf.keras.layers.Dense(10, activation='softmax'))

    # compile the model
    model.compile(optimizer=tf.keras.optimizers.RMSprop(),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# use tf.data to batch and shuffle the dataset
train_ds = tf.data.Dataset.from_tensor_slices(
    (x_train, y_train)).shuffle(len(x_train)).repeat().batch(32)
test_ds = tf.data.Dataset.from_tensor_slices((x_test, y_test)).batch(32)

# build model
model = model_fn()
```

```
# early stopping
checkpoint = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    mode='auto',
    patience=5)

# assign callback
callbacks = [checkpoint]

# train the model
history = model.fit(train_ds, epochs=10,
                    steps_per_epoch=100,
                    validation_data=test_ds,
                    callbacks=callbacks)

# evaluate the model
score = model.evaluate(test_ds)
print('Test loss: {:.2f} \nTest accuracy: {:.2f}%'.format(score[0],
score[1]*100))
```

With early stopping applied to the preceding code, the training will stop once there is no improvement to the loss on the validation dataset. The **'patience'** parameter in the EarlyStopping method represents the number of epochs with no improvement, after which training will be stopped.

This chapter surveys some techniques to tackle the problem of overfitting when training with a deep neural network. In the next chapter, we will discuss on convolutional neural networks for building predictive models for computer vision use cases such as image recognition with TensorFlow 2.0.